Week 2 Project 3: Student performance and Attendance Analyzer.

Dataset Name: /content/sample_data/Student Academic Performance.csv

- Student_ID.
- Subject.
- Marks.
- Attendance_Percentage.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("/content/sample_data/Student Academic Performance.csv")
display(df.head())
```

	Name	Gender	Phone Number	State Of Birth	Email	Student ID	Qualification	Course	Semester	GPA	CGPA	Rais Han
0	Lum Meu Mel	F	010-981 7433	Penang	meumel32@yahoo.com	21PMD30001	Degree	Hotel Management	S	3.958	3.817	
1	Eric Hew Law Toh	M	019-752 8050	Kedah	law03toh@hotmail.com	21PMD30002	Diploma	Accounting	F	3.065	3.182	
2	Hao Lin Min	F	019-455 0808	Perak	hoa32@crooks.org	21PMD30003	Degree	Information Technology	S	2.984	3.091	
3	Cheng Zheng Xun	M	016-375 8448	Penang	zheng1xun@gmail.com	21PMD30004	Diploma	Computer Science	F	2.820	2.606	
4	Eunice Cheong En Hui	F	014-550 5401	Kedah	enhui12@gmail.com	21PMD30005	Degree	Business	F	3.621	3.752	

```
print("Dataset Shape:", df.shape)
print("\nColumns:\n", df.columns)
print("\nData Types:\n", df.dtypes)
print("\nFirst 5 Records:\n", df.head())
```

```
Data Types:
 Name                      object
Gender                     object
Phone Number               object
State Of Birth             object
Email                      object
Student ID                 object
Qualification              object
Course                     object
Semester                   object
GPA                       float64
CGPA                      float64
Raised Hands               object
Visited Resource          float64
Announcements Visited     float64
Discussion                float64
Absence Days Under-7      float64
Unnamed: 16               float64
Unnamed: 17               float64
Unnamed: 18               float64
dtype: object

First 5 Records:
```

```
1  law03toh@hotmail.com  21PMD30002       Diploma              Accounting
```

```
2      hoa32@crooks.org  21PMD30003        Degree  Information Technology
3   zheng1xun@gmail.com  21PMD30004       Diploma        Computer Science
4     enhui12@gmail.com  21PMD30005        Degree                Business

  Semester    GPA   CGPA Raised Hands  Visited Resource  \
0        S  3.958  3.817           43              47.0
1        F  3.065  3.182            9              15.0
2        S  2.984  3.091            7              12.0
3        F  2.820  2.606            8               6.0
4        F  3.621  3.752           31              29.0

   Announcements Visited  Discussion  Absence Days Under-7  Unnamed: 16  \
0                   42.0        36.0                   1.0          NaN
1                   13.0        10.0                   0.0          NaN
2                   11.0         9.0                   0.0          NaN
3                    7.0         8.0                   0.0          NaN
4                   28.0        29.0                   1.0          NaN

   Unnamed: 17  Unnamed: 18
0          NaN          NaN
1          NaN          NaN
2          NaN          NaN
3          NaN          NaN
4          NaN          NaN
```

```
print("\nMissing Values:\n", df.isnull().sum())
```

```
Missing Values:
 Name                       2
Gender                      2
Phone Number                2
State Of Birth              2
Email                       2
Student ID                  2
Qualification               2
Course                      2
Semester                    2
GPA                         2
CGPA                        2
Raised Hands                2
Visited Resource            2
Announcements Visited       2
Discussion                  2
Absence Days Under-7        2
Unnamed: 16               102
Unnamed: 17               102
Unnamed: 18               102
dtype: int64
```

```
# The 'Marks' column does not exist in the DataFrame. If it did, the correct code for filling missing values would be:
# df['Marks'] = df['Marks'].fillna(df['Marks'].mean())
```

```
# The 'Attendance_Percentage' column does not exist in the DataFrame. If it did, the correct code for filling missing value
# df['Attendance_Percentage'] = df['Attendance_Percentage'].fillna(
#     df['Attendance_Percentage'].mean()
# )
```

```
print("\nOverall CGPA Statistics:")
print(df['CGPA'].describe())
```

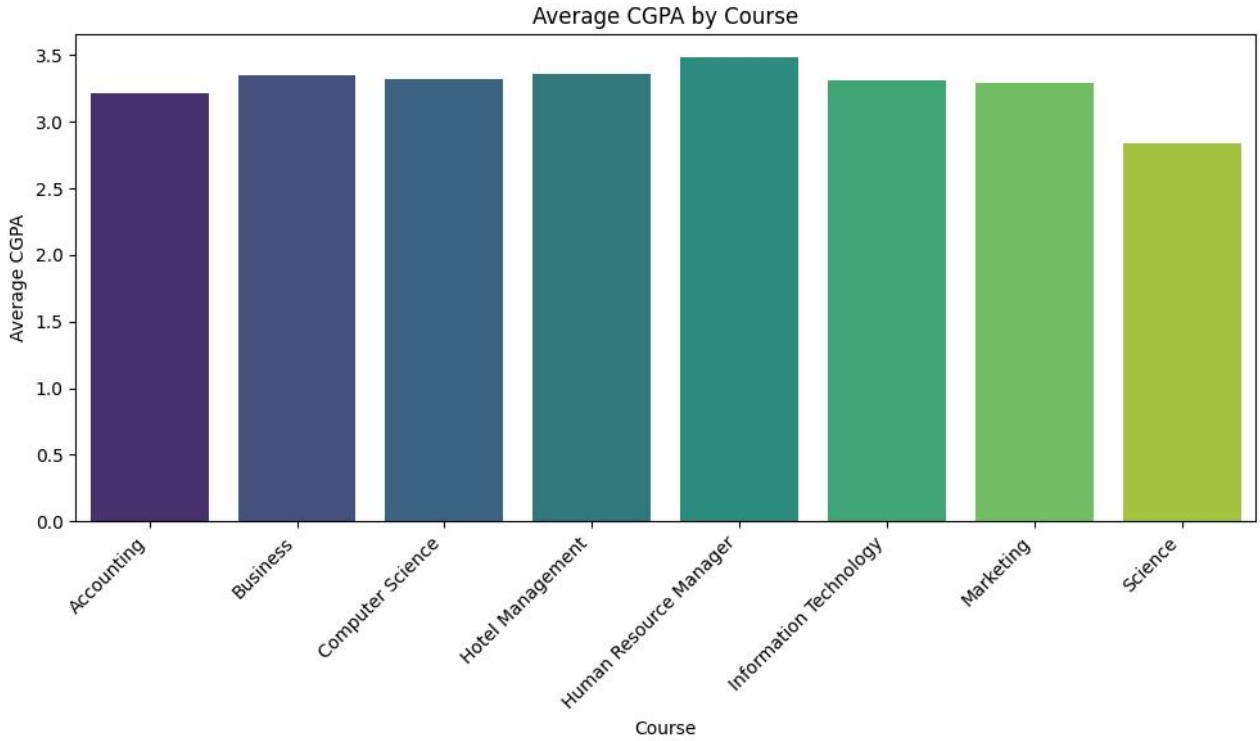
```
Overall CGPA Statistics:
count    100.000000
mean       3.297080
std        0.386223
min        2.319000
25%        3.086750
50%        3.335000
75%        3.562000
max        3.939000
Name: CGPA, dtype: float64
```

Before calculating 'Attendance_Percentage', let's handle any missing values in the 'Absence Days Under-7' column. We'll fill them with the mean of the column.

```
plt.figure(figsize=(10, 6))
sns.barplot(x=course_cgpa.index, y=course_cgpa.values, palette='viridis')
plt.title('Average CGPA by Course')
plt.xlabel('Course')
plt.ylabel('Average CGPA')
plt.xticks(rotation=45, ha='right')
```

```
plt.tight_layout()
plt.show()
```

```
/tmp/ipython-input-342933509.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.barplot(x=course_cgpa.index, y=course_cgpa.values, palette='viridis')
```




```
course_cgpa = df.groupby('Course')['CGPA'].mean()
print("\nAverage CGPA by Course:\n", course_cgpa)
```

```
Average CGPA by Course:
 Course
Accounting                3.213917
Business                  3.344700
Computer Science          3.319125
Hotel Management          3.354400
Human Resource Manager    3.480571
Information Technology    3.308600
Marketing                 3.291500
Science                   2.839750
Name: CGPA, dtype: float64
```

```
df['Absence Days Under-7'] = df['Absence Days Under-7'].fillna(df['Absence Days Under-7'].mean())
print("Missing values in 'Absence Days Under-7' after filling:")
print(df['Absence Days Under-7'].isnull().sum())
```

```
Missing values in 'Absence Days Under-7' after filling:
0
```

Now, we will calculate the 'Attendance_Percentage'. We'll assume that 'Absence Days Under-7' indicates absences within a total period of 7 days. The formula used will be `(7 - Absence Days) / 7 * 100`.

```
total_days_for_attendance = 7
df['Attendance_Percentage'] = ((total_days_for_attendance - df['Absence Days Under-7']) / total_days_for_attendance) * 100

display(df[['Absence Days Under-7', 'Attendance_Percentage']].head())
```

	Absence Days Under-7	Attendance_Percentage
0	1.0	85.714286
1	0.0	100.000000
2	0.0	100.000000
3	0.0	100.000000
4	1.0	85.714286

Now that we have created the 'Attendance_Percentage' column, you can use the original function to categorize attendance:

```
def attendance_category(att):
    if att >= 85:
        return 'High'
    elif att >= 65:
        return 'Medium'
    else:
        return 'Low'

df['Attendance_Category'] = df['Attendance_Percentage'].apply(attendance_category)
display(df[['Attendance_Percentage', 'Attendance_Category']].head())
```

	Attendance_Percentage	Attendance_Category
0	85.714286	High
1	100.000000	High
2	100.000000	High
3	100.000000	High
4	85.714286	High

```
attendance_performance = df.groupby('Attendance_Category')['GPA'].mean()
print("\nAverage GPA by Attendance Category:\n", attendance_performance)
```

```
Average GPA by Attendance Category:
 Attendance_Category
High    3.35189
Name: GPA, dtype: float64
```

```
print("Distribution of Attendance_Percentage:\n")
print(df['Attendance_Percentage'].value_counts().sort_index())
```

```
Distribution of Attendance_Percentage:

Attendance_Percentage
85.714286     74
89.428571      2
100.000000    26
Name: count, dtype: int64
```

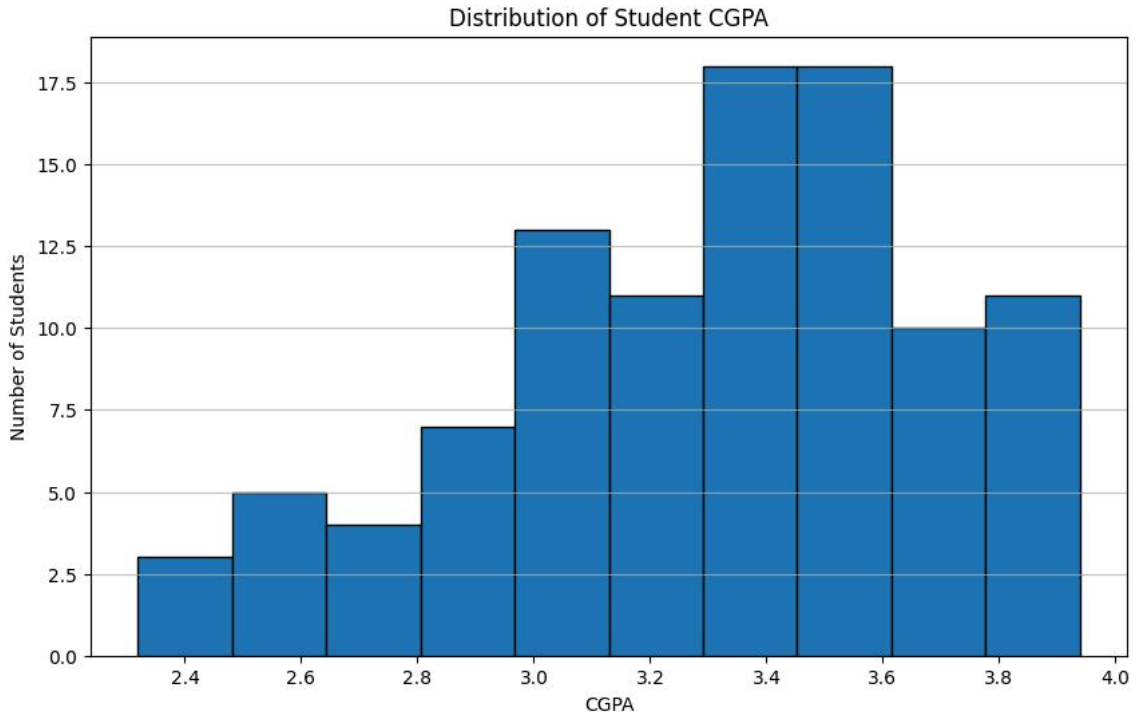
```
gpa_by_attendance = df.groupby('Attendance_Category')['GPA'].mean()
print("\nAverage GPA by Attendance Category:\n", gpa_by_attendance)
```

```
Average GPA by Attendance Category:
 Attendance_Category
High    3.35189
Name: GPA, dtype: float64
```

```
print("Available columns in df:")
print(df.columns)
```

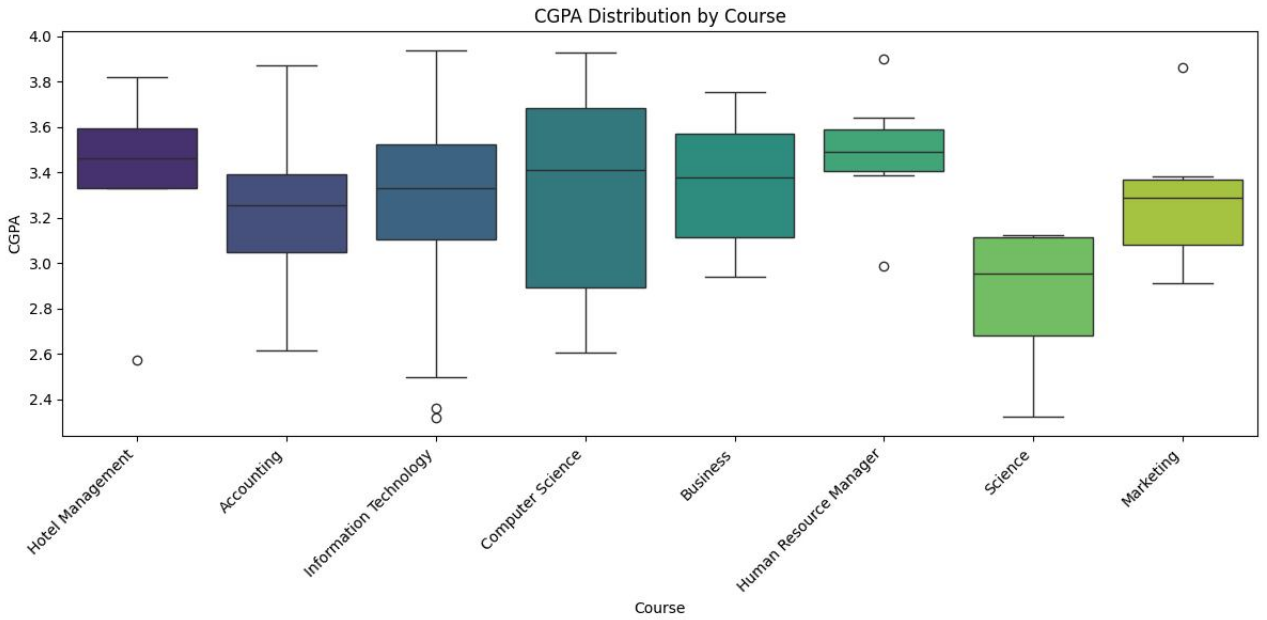
```
Available columns in df:
Index(['Name', 'Gender', 'Phone Number', 'State Of Birth', 'Email',
       'Student ID', 'Qualification', 'Course', 'Semester', 'GPA', 'CGPA',
       'Raised Hands', 'Visited Resource', 'Announcements Visited',
       'Discussion', 'Absence Days Under-7', 'Unnamed: 16', 'Unnamed: 17',
       'Unnamed: 18', 'Attendance_Percentage', 'Attendance_Category'],
      dtype='object')
```

```
plt.figure(figsize=(10, 6))
plt.hist(df['CGPA'], bins=10, edgecolor='black')
plt.title("Distribution of Student CGPA")
plt.xlabel("CGPA")
plt.ylabel("Number of Students")
plt.grid(axis='y', alpha=0.75)
plt.show()
```



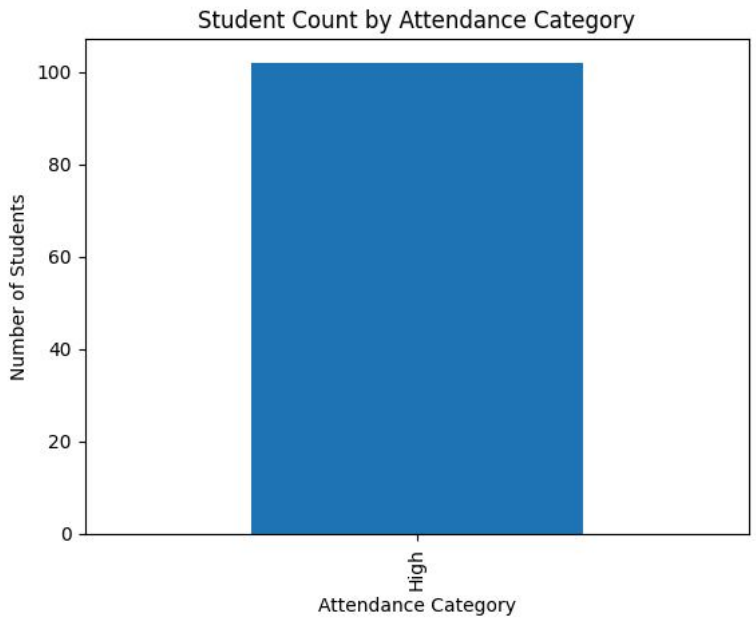

```
plt.figure(figsize=(12, 6))
sns.boxplot(x='Course', y='CGPA', data=df, palette='viridis')
plt.title("CGPA Distribution by Course")
plt.xlabel("Course")
plt.ylabel("CGPA")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
/tmp/ipython-input-2033862197.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.boxplot(x='Course', y='CGPA', data=df, palette='viridis')
```




```
plt.figure()
df['Attendance_Category'].value_counts().plot(kind='bar')
plt.title("Student Count by Attendance Category")
plt.xlabel("Attendance Category")
```

```
plt.ylabel("Number of Students")
plt.show()
```



- Loading data using pandas.
- Checking dataset structure and data type.
- Handling missing values using fillna().
- Grouping data using groupby().
- Creating attendance categories using conditional logic.
- Visualizing data using charts.